

PROFEPI

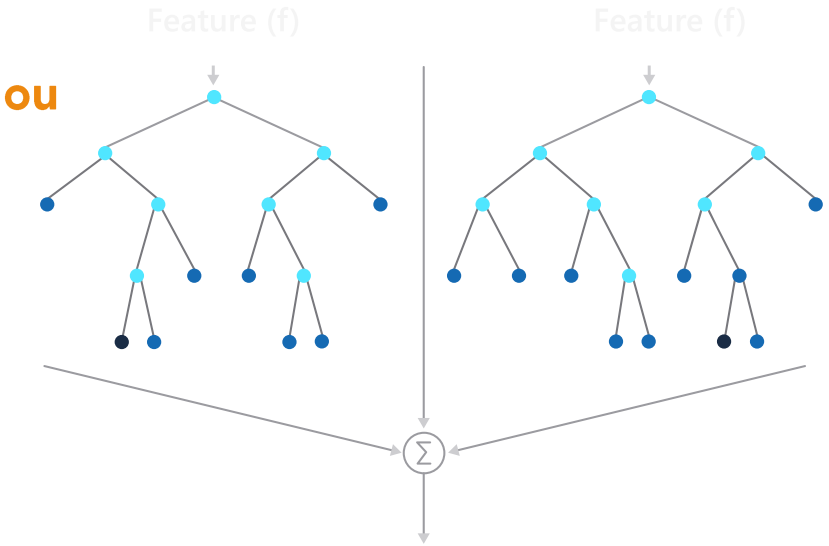
Introdução à inteligência artificial para predições em vigilância em saúde e ambiente

Algoritmos baseados em árvore

O que são algoritmos?

Sequência de instruções bem definidas que resolvem um problema ou realizam uma tarefa.

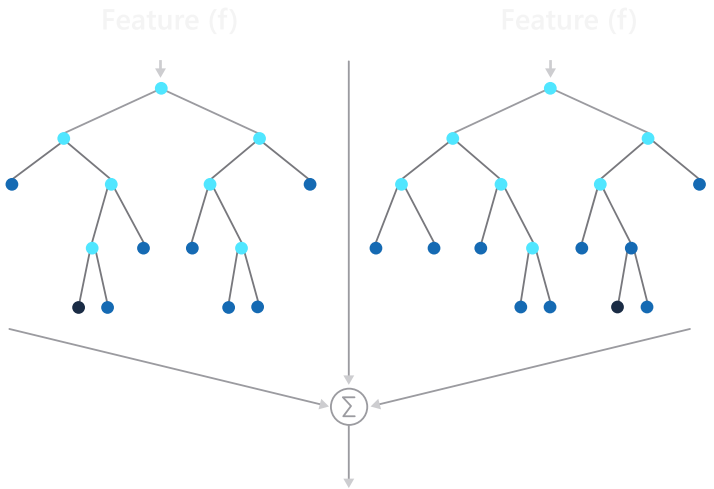
Coração do processo de aprendizado: eles ajustam os modelos a partir dos dados.



O que são algoritmos?

Na vigilância em saúde e ambiente, os algoritmos ajudam a:

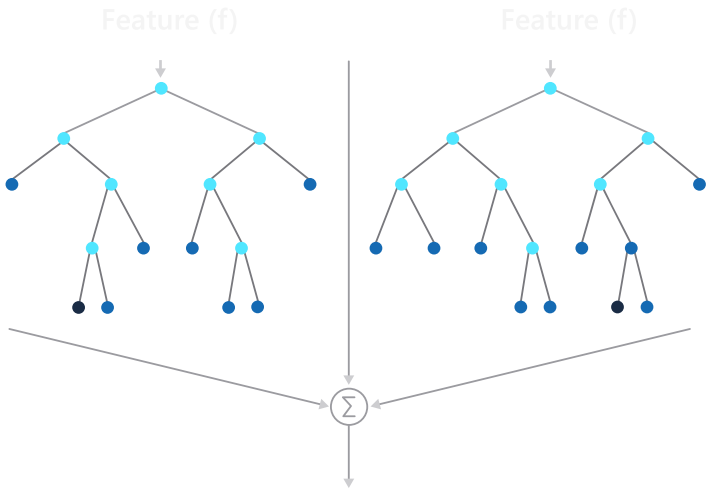
- Prever surtos e epidemias.
- Antecipar riscos ambientais
- Detectar padrões em grandes bases de dados de notificação, mortalidade ou clima.



O que são algoritmos?

Na vigilância em saúde e ambiente, os algoritmos ajudam a:

- Prever surtos e epidemias.
- Antecipar riscos ambientais
- Detectar padrões em grandes bases de dados de notificação, mortalidade ou clima.



Algoritmos podem ser treinados para prever o número de internações por doenças respiratórias com base em dados de poluição do ar, temperatura, umidade e dias da semana.

Por que existem tantos algoritmos diferentes?

- Aprende de uma forma distinta e tem diferentes pontos fortes e fracos.
- Alguns funcionam melhor com dados tabulares, outros com imagens, texto livre ou dados temporais.
- Há algoritmos mais precisos, outros mais interpretáveis, e alguns mais rápidos ou eficientes.
- A escolha depende do problema, dos dados disponíveis e dos objetivos da aplicação



Por que existem tantos algoritmos diferentes?

- Aprende de uma forma distinta e tem diferentes pontos fortes e fracos.
- Alguns funcionam melhor com dados tabulares, outros com imagens, texto livre ou dados temporais.
- Há algoritmos mais precisos, outros mais interpretáveis, e alguns mais rápidos ou eficientes.
- A escolha depende do problema, dos dados disponíveis e dos objetivos da aplicação



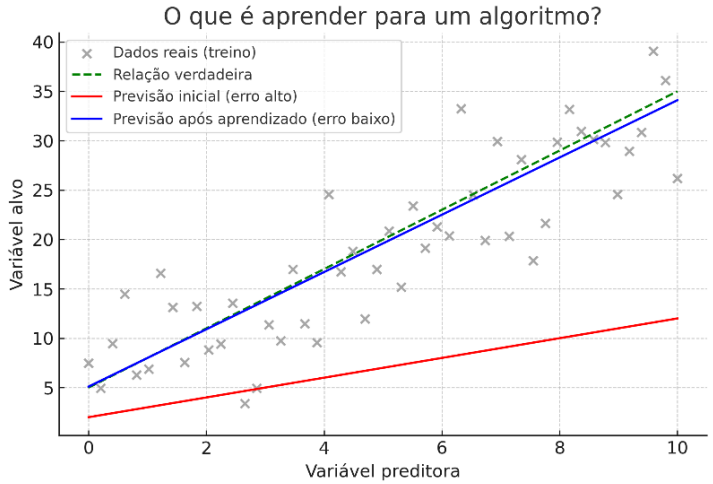
Em vigilância em saúde e ambiente, é importante equilibrar:

- **Precisão do modelo**
- **Facilidade de explicação para a tomada de decisão**
- **Viabilidade computacional**

O que é "aprender" para um algoritmo?

Ajustar seus parâmetros internos para minimizar o erro entre as previsões e os valores reais.

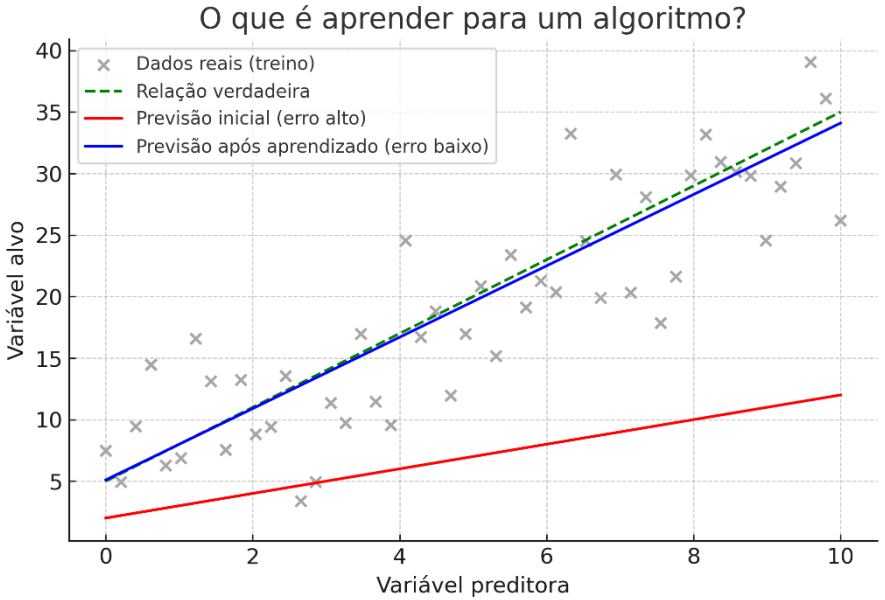
O algoritmo busca generalizar: fazer boas previsões em dados que nunca viu antes.



- Função de perda (mede o erro)
- Método de otimização (como o gradient descent)
- Processo chamado validação cruzada, para evitar que o algoritmo apenas memorize os dados.

O que é "aprender" para um algoritmo?

Em aplicações de vigilância em saúde e ambiente, isso é crucial.
Não basta o modelo acertar o passado, ele precisa ser confiável para antecipar problemas no futuro, como surtos, eventos climáticos extremos ou aumento de hospitalizações.

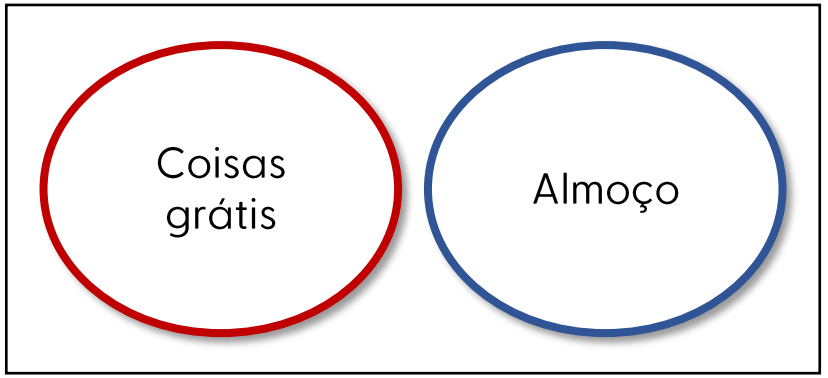


Tipos de algoritmos

	Família	Exemplos	Características
	Baseados em árvore	Árvores de decisão, Random Forest, XGBoost, CatBoost, LightGBM	Interpretáveis, robustos para dados tabulares, lidam bem com variáveis mistas, ótimos para classificação e regressão.
	Baseados em redes neurais	Redes neurais artificiais, Deep Learning, TabPFN	Poderosos para detectar padrões complexos, aplicáveis a diversos tipos de dados, mas menos interpretáveis.

Teorema do "não há almoço grátis"

- Em infinitos conjuntos diferentes de dados, nenhum algoritmo é garantido a priori de ter melhor performance.
- A única forma de saber qual vai ter melhor performance é testar todos.

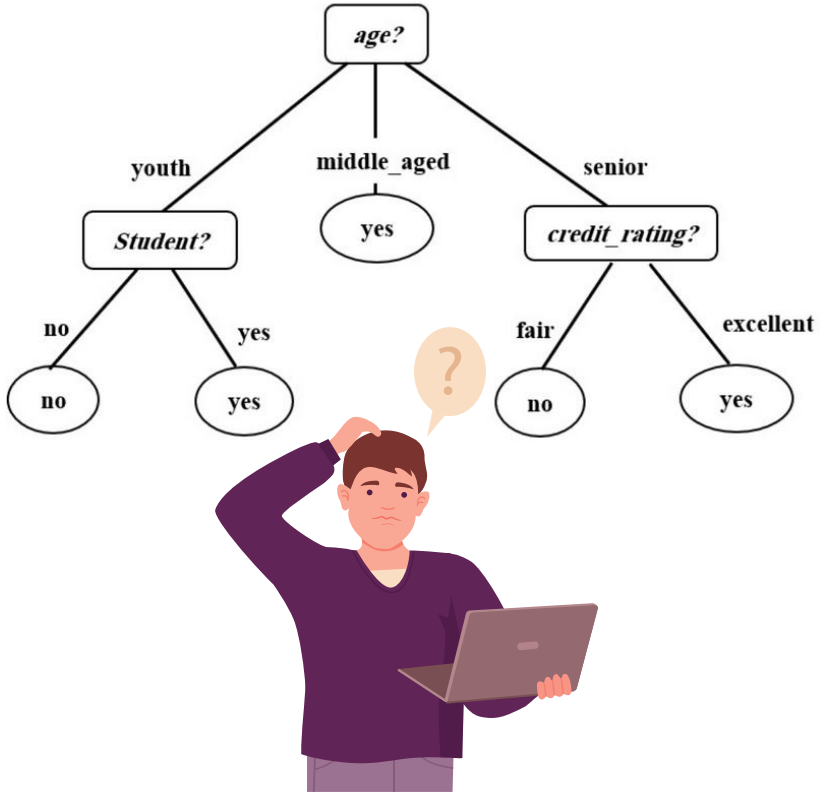


Segredo:
com dados reais, alguns costumam ganhar mais (XGBoost).

Árvore de decisão

Um algoritmo que toma decisões dividindo os dados em ramos, a partir de perguntas sucessivas sobre as variáveis, até chegar a um resultado final.

Componentes principais:
Nós internos → Perguntas baseadas nas variáveis.
Ramos → Caminhos possíveis de resposta.
Folhas → Resultado final (classe ou valor previsto).

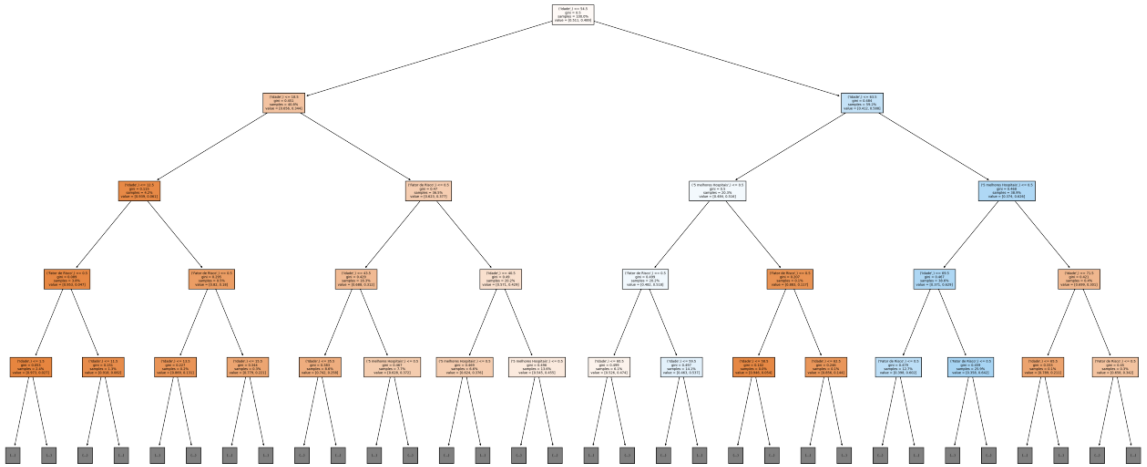


Exemplo: comprar um computador ou não
A predição do modelo mostra que um jovem estudante vai comprar

Como a árvore é construída?

Processo de treinamento:

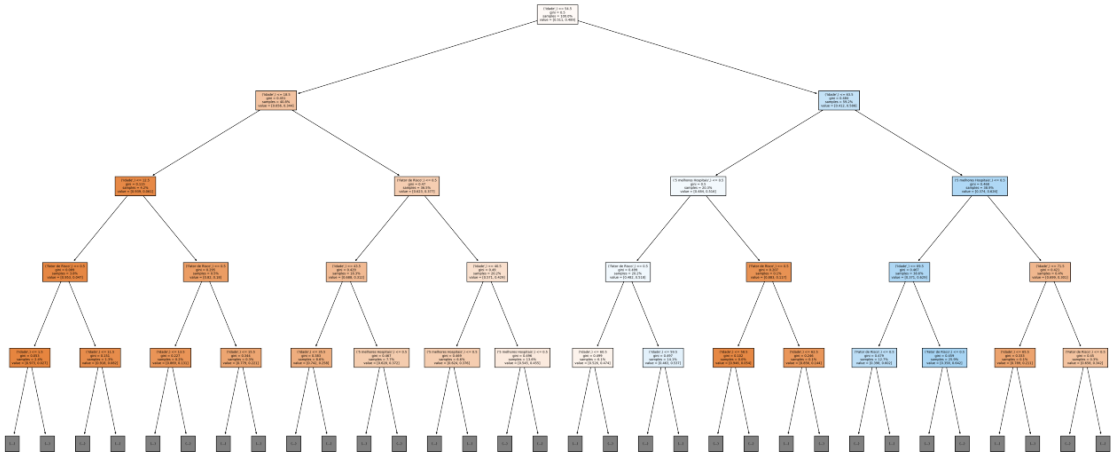
- Começa com todos os dados no nó raiz.
- Para cada variável e ponto de corte, calcula-se o critério de separação.
- Escolhe-se a variável e o ponto que melhor separa os dados em termos do desfecho.
- Divide-se o conjunto em dois ramos e repete-se o processo em cada subconjunto (recursão).



Como a árvore é construída?

Critérios comuns:

Gini → mede a impureza (classificação).
Entropia → mede a incerteza (classificação).
Erro quadrático médio → usado em problemas de regressão.



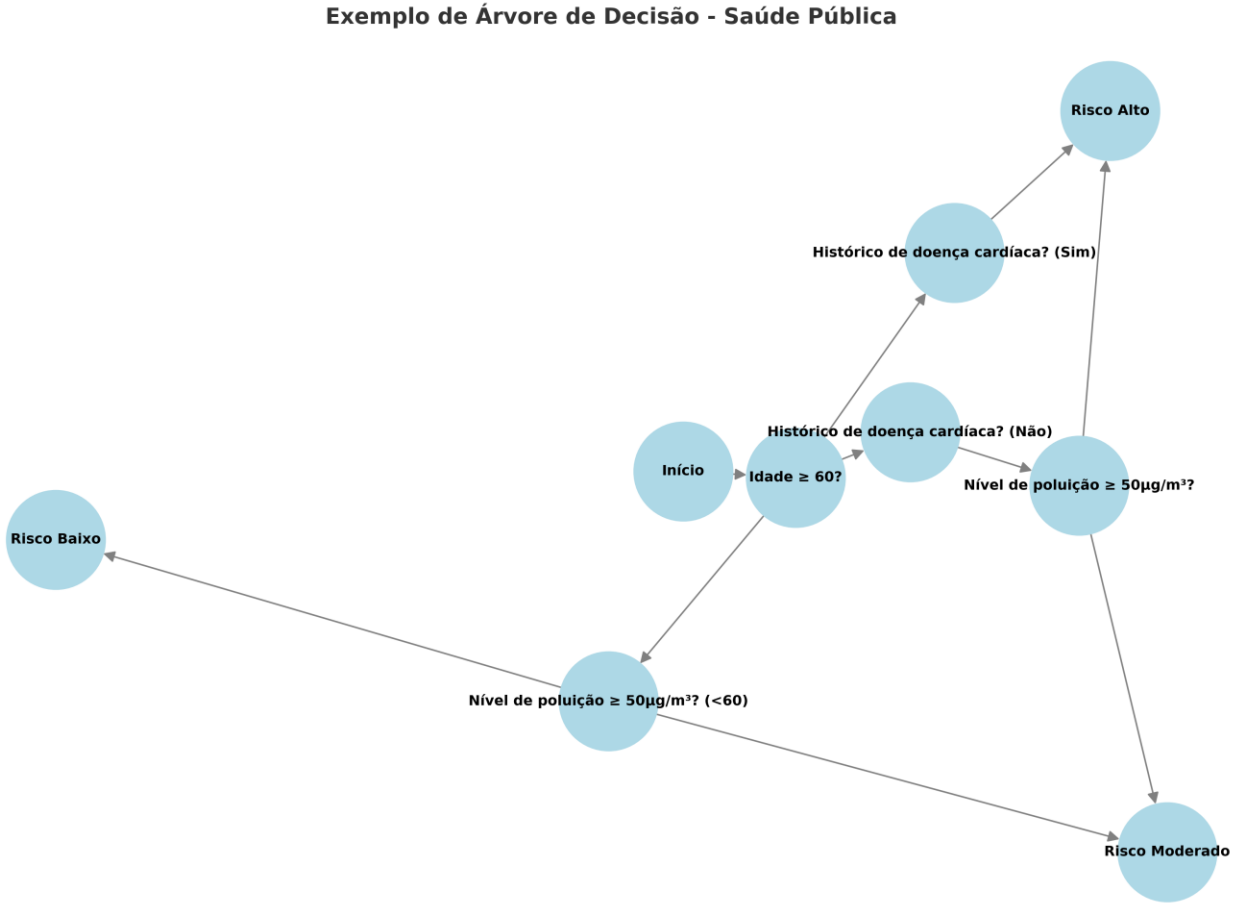
Parada do crescimento:
Profundidade máxima definida.
Número mínimo de observações por nó.
Impureza mínima para continuar dividindo.

Idade ≥ 60 anos?

- Sim, vamos para a próxima pergunta: *'Histórico de doença cardíaca?'*.
  - Se sim, classificamos diretamente como **Risco Alto**.
  - Se não, verificamos o *'Nível de poluição'*. Se for ≥ 50 µg/m³, também é **Risco Alto**.

Se idade < 60 anos, verificamos o 'Nível de poluição'.

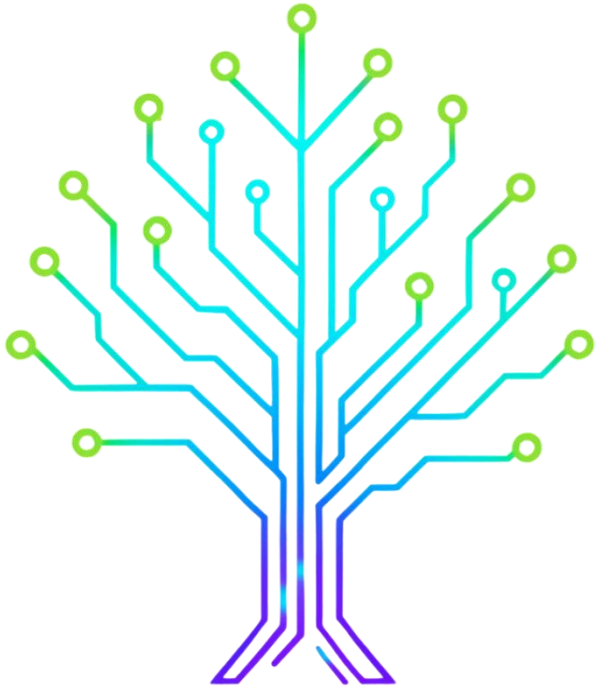
- Se for < 50 µg/m³, classificamos como **Risco Baixo**.
- Se for ≥ 50 µg/m³, classificamos como **Risco Moderado**.



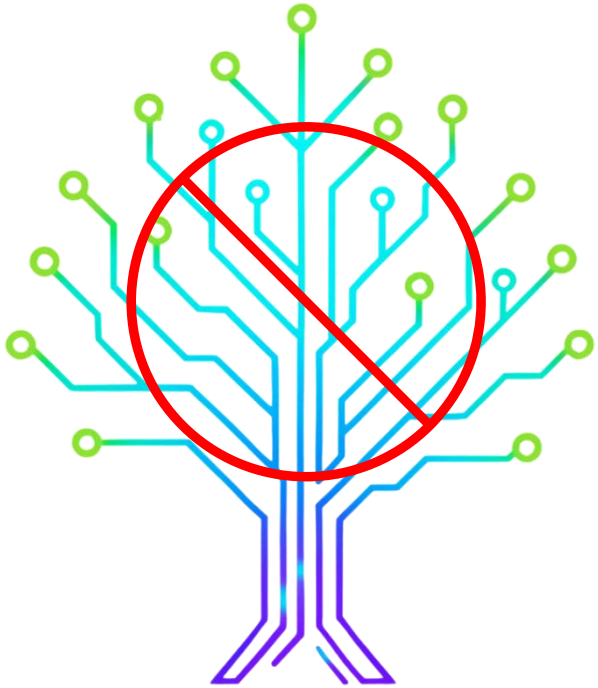
Vantagens

- Fáceis de interpretar e explicar → resultado em formato de regras lógicas.
- Não exigem grande pré-processamento → funcionam bem com variáveis numéricas e categóricas.
- Robustas a outliers → valores extremos têm pouco impacto nas divisões.
- Lidam bem com dados faltantes
- Baixo custo computacional para árvores pequenas e médias.
- Úteis para seleção de variáveis





Limitações



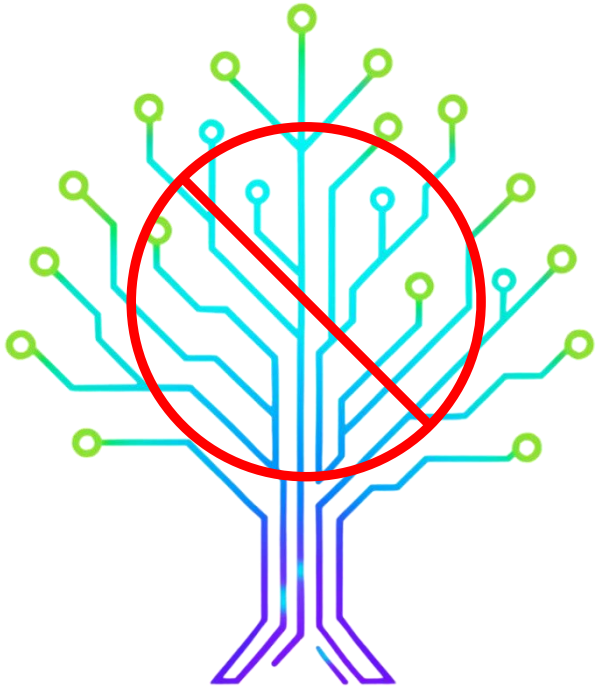
•Alta sensibilidade a pequenas variações nos dados (instabilidade).

•Podem gerar fronteiras de decisão pouco suaves.

•Desempenho pode ser inferior a outros algoritmos em problemas complexos.

Limitações

- Pode levar a sobreajuste do modelo
- Solução: Ensemble
- Ensemble = várias algoritmos combinados (método que generaliza bem)
- Existem formas diferentes de combinar as árvores
- GBDT é uma delas



REALIDADE							
PREDITOR 1							
PREDITOR 2							
PREDITOR 3							
PREDITOR 4							
PREDITOR 5							
COMITÊ							

Limitações

Maior problema das árvores de decisão simples são a sua forte tendência ao sobreajuste (sensíveis a pequenas variações nos dados).

Uma solução é o uso de bagging (bootstrap aggregation).

Bagging: selecionar amostras aleatórias dos dados de treino.

Cada observação tem a mesma probabilidade de ser sorteada (com reposição).



Probabilidade de cada observação ser escolhida em uma rodada é de aproximadamente 63,2%

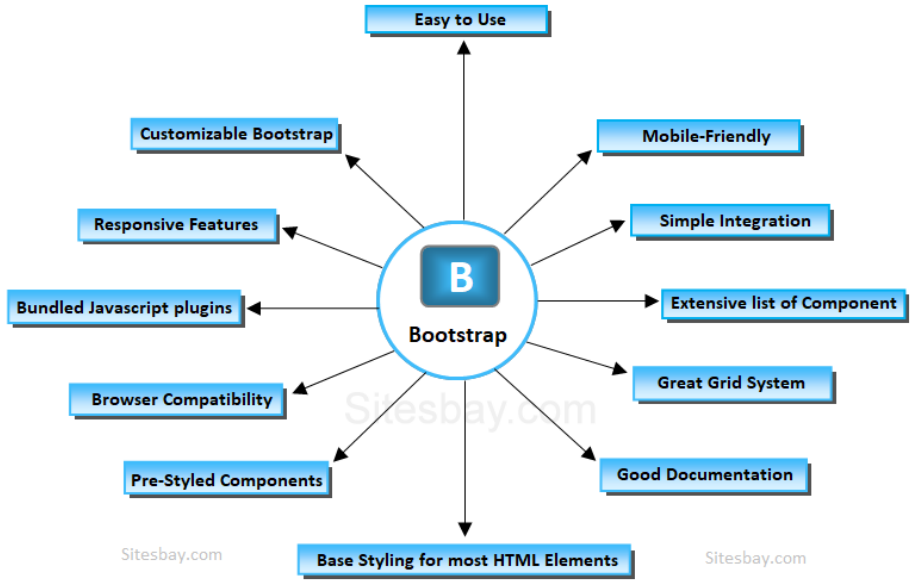
Cada amostra de bootstrap é utilizada para definir a sua árvore.

Árvores são definidas sem regularização (complexas).

No caso de regressão, a predição final de uma observação é a sua média de todas as árvores.

No caso de classificação, a predição final de uma observação é a sua categoria mais predita em todas as árvores (probabilidade é a proporção de votos).

É considerada uma técnica de ensemble por agregar os resultados de vários algoritmos (de árvore).



- O problema do uso de bagging é o custo computacional (porém como as árvores são independentes é fácil paralelizar a análise).
- Pode envolver centenas ou milhares de árvores.
- Uma outra limitação importante é que a facilidade de interpretação, um dos principais pontos fortes das árvores de decisão, foi perdida.

Data-driven advice for applying machine learning to bioinformatics

Randal S. Olson, William La Cava, Zairah Mustahsan, Akshay Varik, Jason H. Moore.
arXiv, 2018.

Análise da performance preditiva (acurácia de VC) de 13 algoritmos em 165 bancos de dados diferentes.

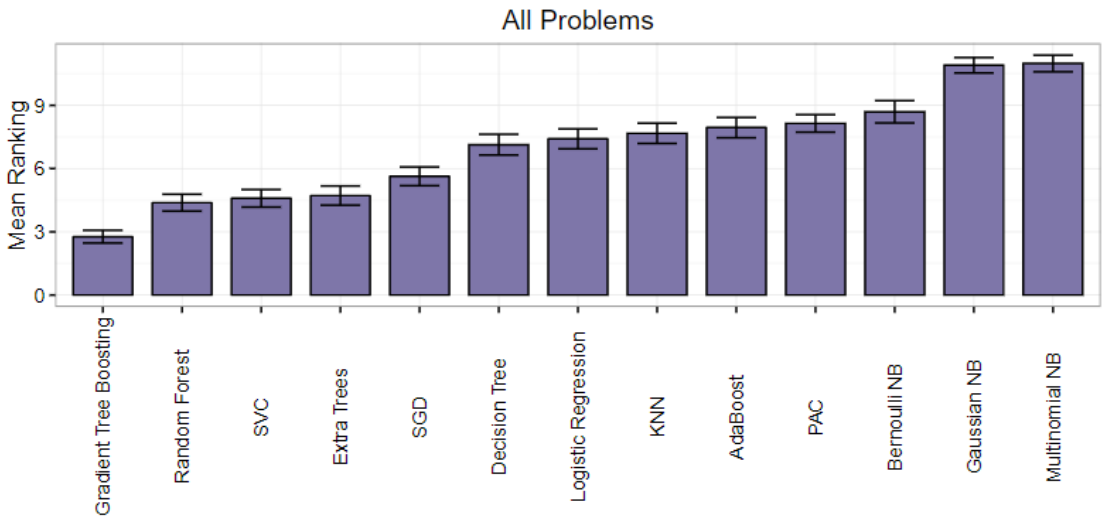


Fig. 1. Average ranking of the ML algorithms over all datasets. Error bars indicate the 95% confidence interval.

Data-driven advice for applying machine learning to bioinformatics

Randal S. Olson, William La Cava, Zairah Mustahsan, Akshay Varik, Jason H. Moore.
arXiv, 2018.

Ganhos de performance ao selecionar hiperparâmetros via validação cruzada de 10-folds (vs. usar o hiperparâmetro default do scikit-learn): média 3-4%, em alguns casos 50%.

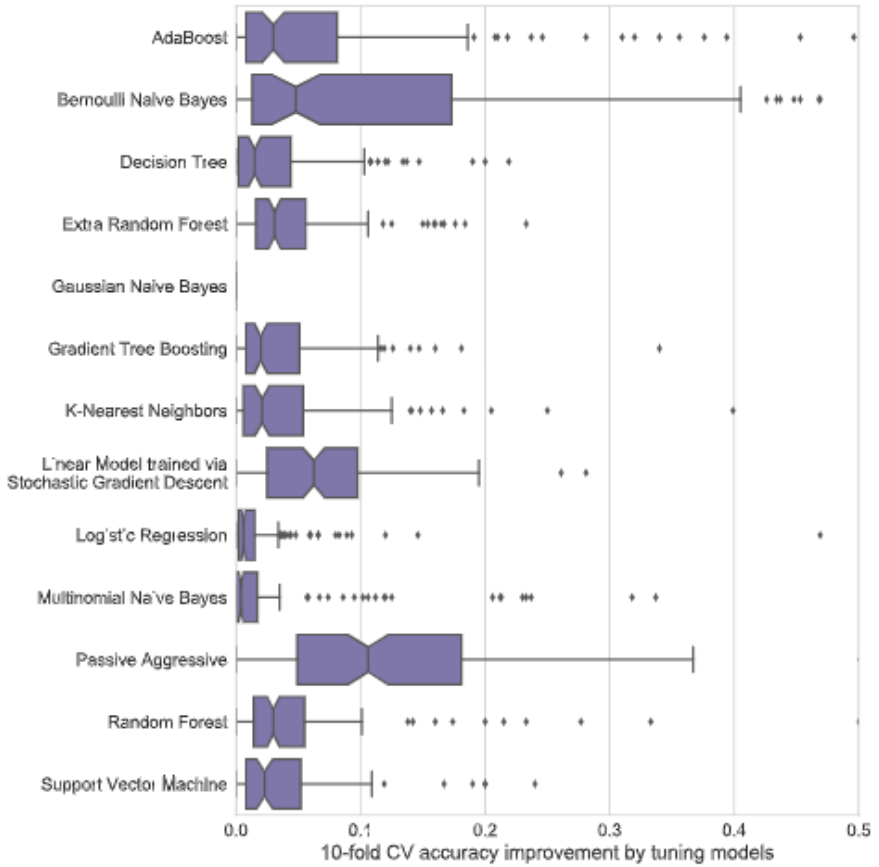


Fig 3. Improvement in 10-folf CV accuracy by tuning each ML algorithm's parameters intead of using the default parameters from scikit-learn

Ganhos de performance ao selecionar hiperparâmetros via validação cruzada de 10-folds (vs. usar o hiperparâmetro default do scikit-learn): média 3-4%, em alguns casos 50%.

Data-driven advice for applying machine learning to bioinformatics

Randal S. Olson, William La Cava, Zairah Mustahsan, Akshay Varik, Jason H. Moore.
arXiv, 2018.

Quando combinamos duas estratégias: Selecionar o melhor algoritmo para o problema e ajustar seus hiperparâmetros usando validação cruzada de 10 folds, os ganhos de desempenho são ainda mais expressivos.

Em média, o aumento de acurácia chega a 20%, e em alguns casos ultrapassa

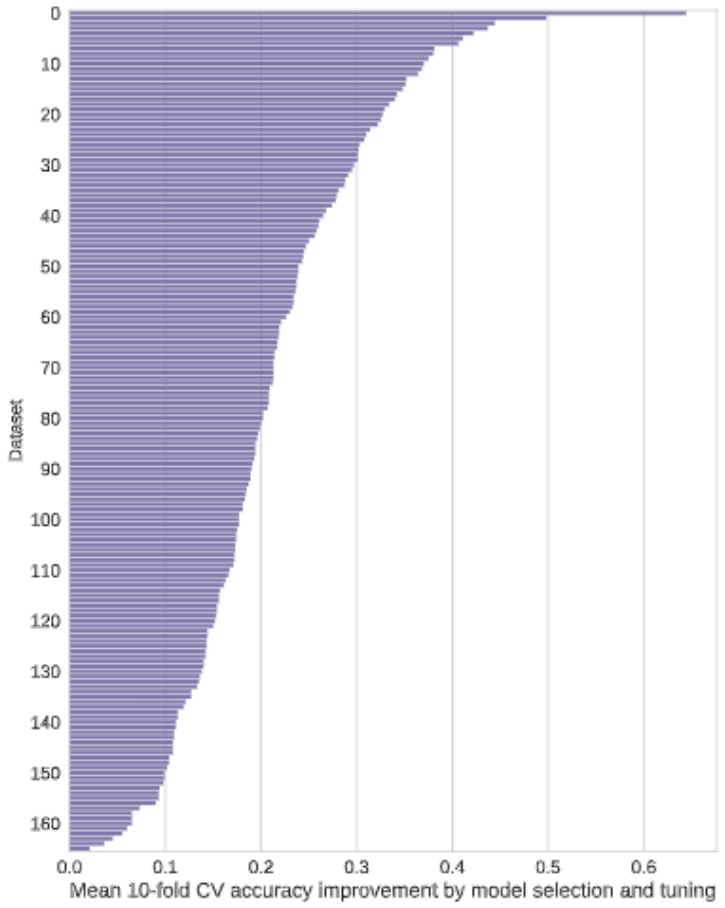
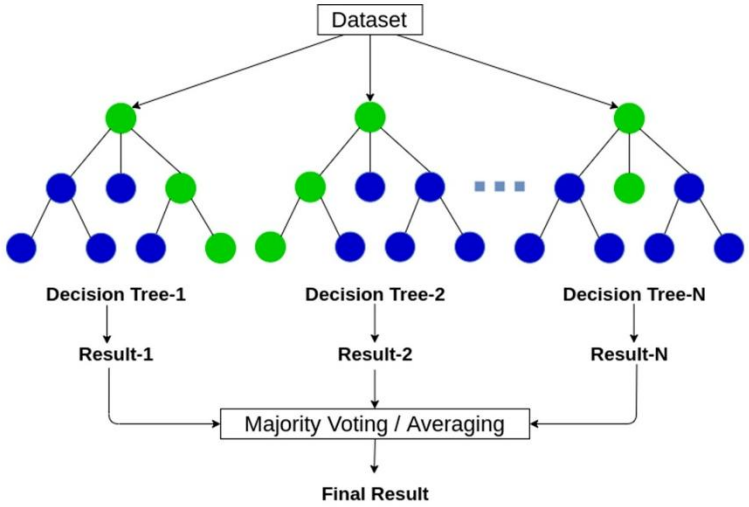


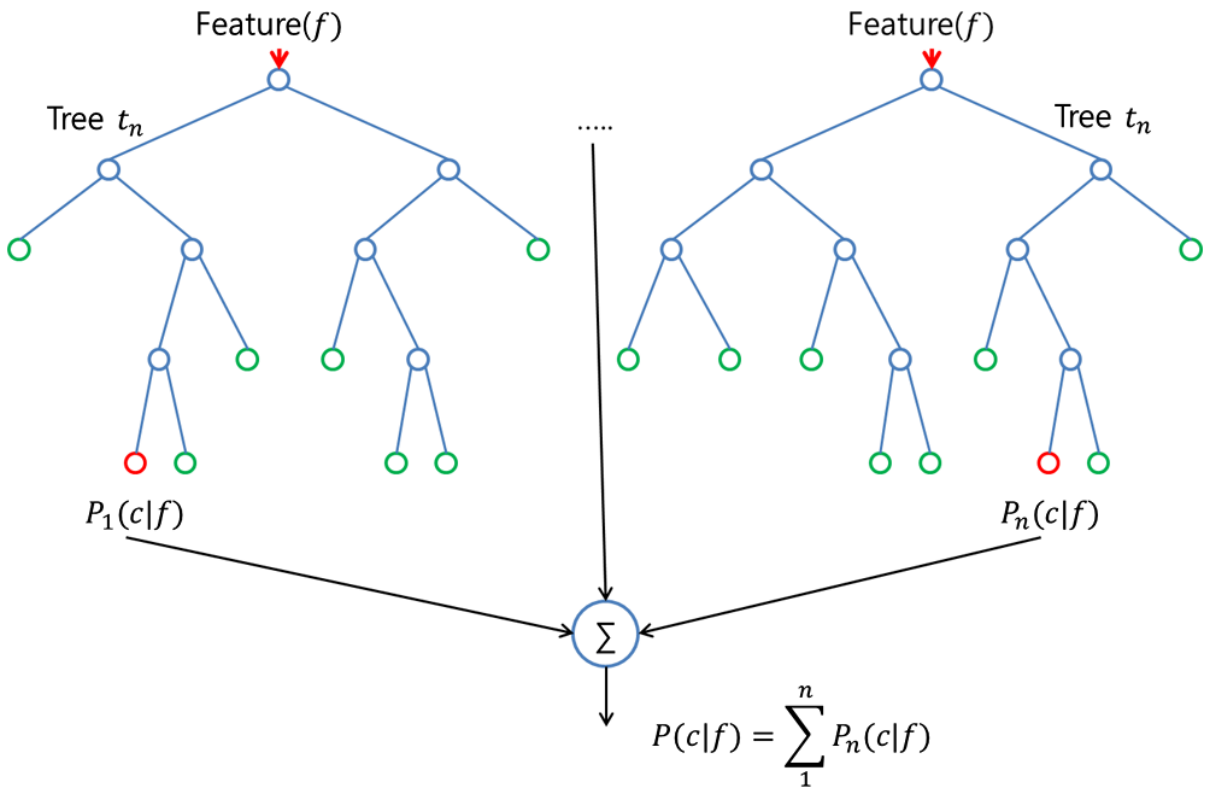
Fig 4 Improvement in 10-fold CV accuracy by model selection and tuning, relative to the average performance on each dataset

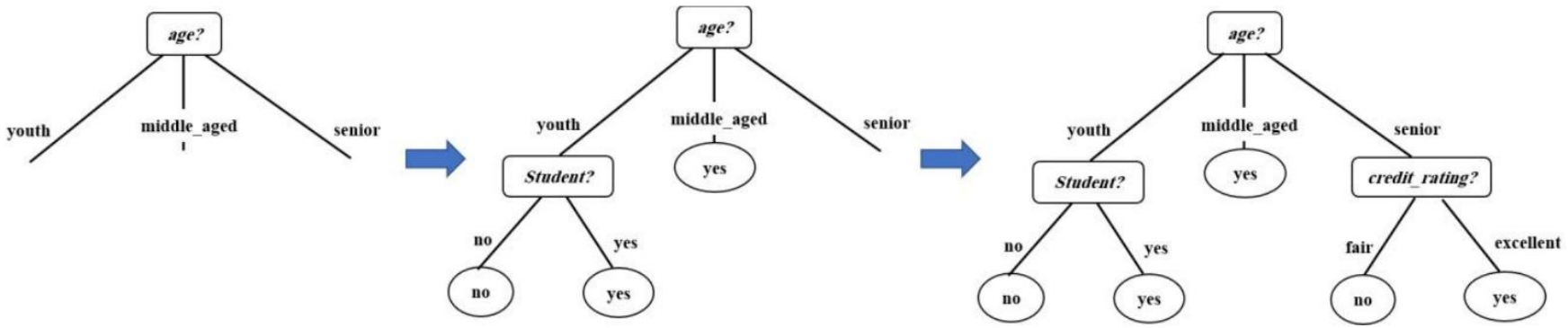
Random Forest

- Algoritmo de *ensemble* que combina várias árvores de decisão.
- Cada árvore é treinada com uma amostra aleatória dos dados (*bagging*).
- O resultado final é dado por votação (classificação) ou média (regressão).
- Alta performance e boa generalização.



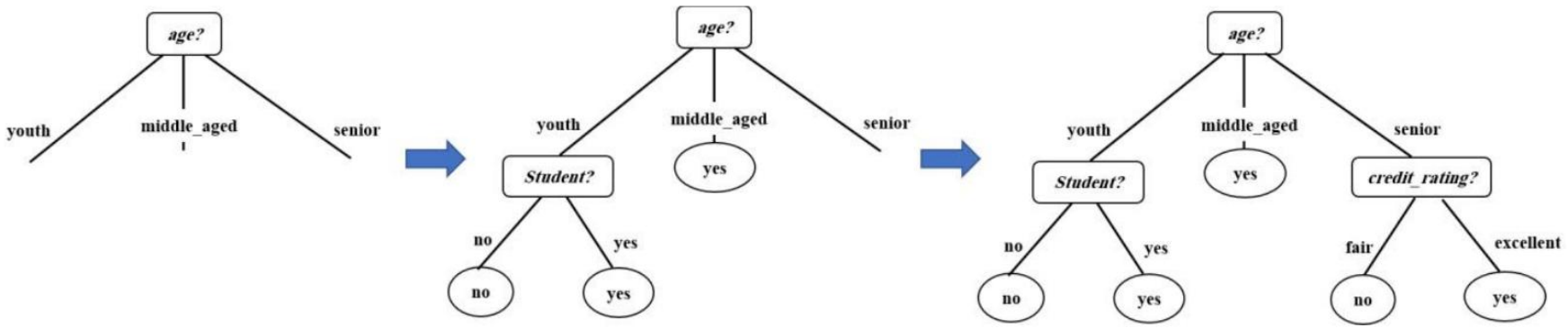
Random Forest





Ideia do GBDT:

- Treina várias árvores sequenciais
- O resultado predito será a soma das predições de todas as árvores
- Treinamento iterativo = constrói as árvores uma por uma



Ideia do GBDT:

- Objetivo do treinamento: minimizar uma função objetivo por meio da divisão repetitiva dos dados para maximizar algum critério até algum limite (ex.: profundidade da árvore).

- A árvore seguinte é construída de modo a minimizar a função de perda, e suas saídas são adicionadas a primeira árvore.

XGboost, LightGBM e CatBoost são os principais representantes do GBDT pois:

- Implementações fáceis (Windows, macOS e Linux)
- Bibliotecas de software de código aberto (C++, Python e R)
- Rápidos de treinar e resultados precisos

CatBoost

Algoritmo de *Gradient Boosting* otimizado desenvolvido pela Yandex

Excelente desempenho pré-processamento com variáveis categóricas

Objetivo:

Melhorar a performance e reduzir o tempo de pré-processamento de dados

CatBoost

- Treina árvores sequenciais como o GBDT.
- Usa um esquema específico de ordenamento das amostras para evitar vazamento de informação (*target leakage*).
- Trabalha com *Ordered Boosting* e *Oblivious Trees* para acelerar cálculos e manter consistência.

CatBoost

- Lida nativamente com variáveis categóricas.
- Reduz overfitting sem necessidade de ajustes complexos.
- Ótima performance em datasets pequenos e grandes.
- Fácil de usar, com implementação em Python, R, C++ e Java.
- Compatível com GPU para acelerar treinamento.

CatBoost

Cenários ideais:

- Modelos com muitas variáveis categóricas (ex.: códigos de doenças, regiões, profissões).
- Bases com mistura de dados categóricos e numéricos.
- Quando se busca um bom equilíbrio entre desempenho e tempo de treino.

CatBoost

Desvantagens:

Tempo de treinamento maior

Em alguns cenários, especialmente com muitos dados numéricos, ele pode ser mais lento que LightGBM ou XGBoost.

Uso de memória

Pode consumir mais memória RAM em datasets muito grandes devido à forma como processa e armazena combinações de variáveis categóricas.

Menor flexibilidade de ajuste

Oferece menos parâmetros para *fine-tuning* comparado ao XGBoost e LightGBM.

CatBoost

Desvantagens:

Comunidade e recursos menores

Quantidade de tutoriais, bibliotecas de terceiros e exemplos práticos ainda é menor que a do XGBoost.

Limitações com dados exclusivamente numéricos

O ganho de desempenho é mais perceptível em datasets com variáveis categóricas; se todos os dados forem numéricos, pode não trazer benefícios significativos frente a outros algoritmos.

XGBoost

XGBoost (*Extreme Gradient Boosting*) é uma implementação otimizada da técnica de *gradient boosting*, muito eficiente em termos de velocidade e desempenho.

Objetivo:

Criar modelos mais rápidos e precisos, utilizando otimizações como processamento paralelo, regularização e manipulação eficiente de memória.

Tipo:

Algoritmo de *boosting* baseado em árvores de decisão.

XGBoost

- Constrói árvores de decisão de forma sequencial, cada nova árvore corrigindo erros da anterior.
- Otimiza usando *gradient descent* e função de perda personalizada.
- Inclui regularização L1 e L2 para reduzir overfitting.
- Suporta dados esparsos e tratamento de valores ausentes nativamente.

XGBoost

Vantagens

- Alta performance em competições e aplicações reais.
- Processamento paralelo e rápido.
- Flexível na escolha de funções de perda.
- Menor risco de overfitting com regularização embutida.

XGBoost

Desvantagens

- Mais complexo de ajustar (muitos hiperparâmetros).
- Pode ser mais lento que Random Forest em datasets muito pequenos.
- Interpretabilidade mais difícil que modelos simples.

LightGBM

- Algoritmo de *gradient boosting* desenvolvido pela Microsoft.
- Otimizado para alta performance e grandes volumes de dados.
- Utiliza abordagem leaf-wise para crescer a árvore, priorizando os ramos que mais reduzem o erro.
- Suporte nativo a variáveis categóricas e paralelismo.

LightGBM

Vantagens

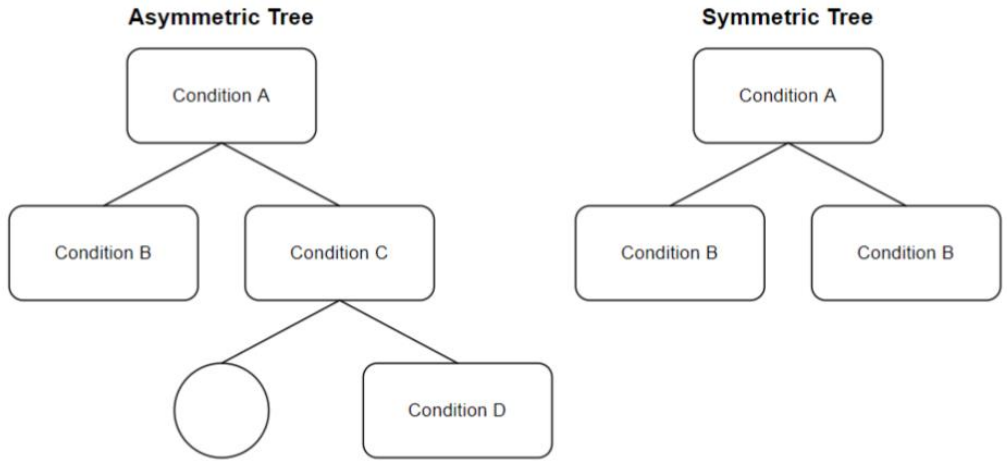
- Muito rápido no treinamento, mesmo com grandes bases.
- Alta acurácia, especialmente em dados tabulares complexos
- Boa eficiência de memória.
- Suporte a GPU

LightGBM

Desvantagens

- Pode gerar *overfitting* se não houver regularização adequada.
- Mais sensível a hiperparâmetros que XGBoost ou Random Forest.

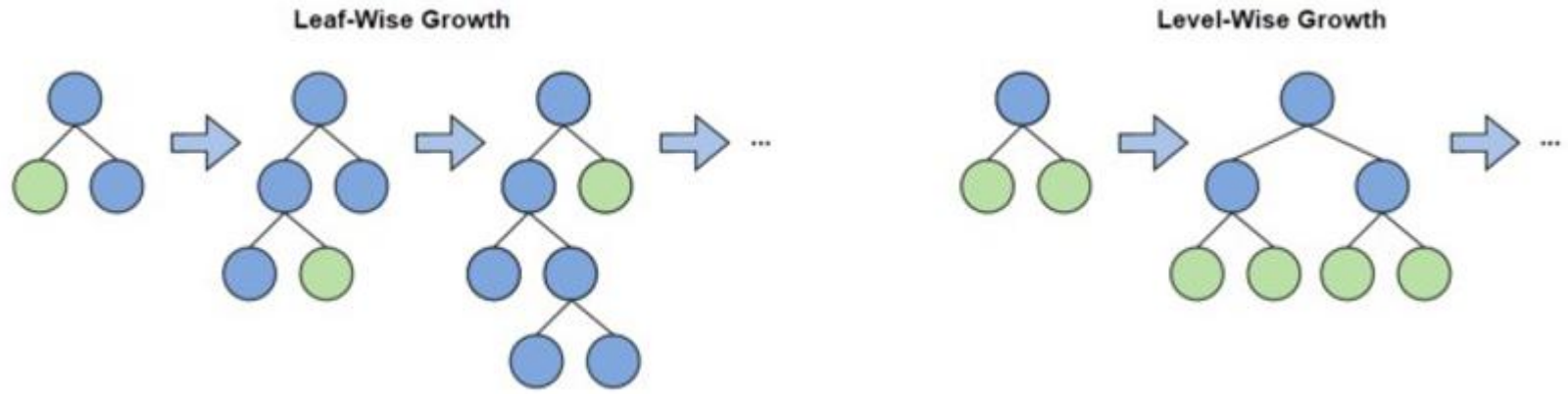
O desenvolvimento da árvore



XGBoost e LightGBM : árvores assimétricas.

Catboost: árvores simétricas (no mesmo nível, todos os nós são iguais).

O desenvolvimento da árvore



O LightGBM cresce as árvoes por folhas (horizontais)
O XGBoost emprega uma estratégia por nível (verticais)

O desenvolvimento da árvore

Estratégia baseada em níveis

Vantagens:

- Mantém a árvore balanceada
- Qualquer árvore construída com essa estratégia pode ser construída com uma estratégia baseada em folhas (o inverso não é verdadeiro)

Estratégia baseada em folhas

Vantagens:

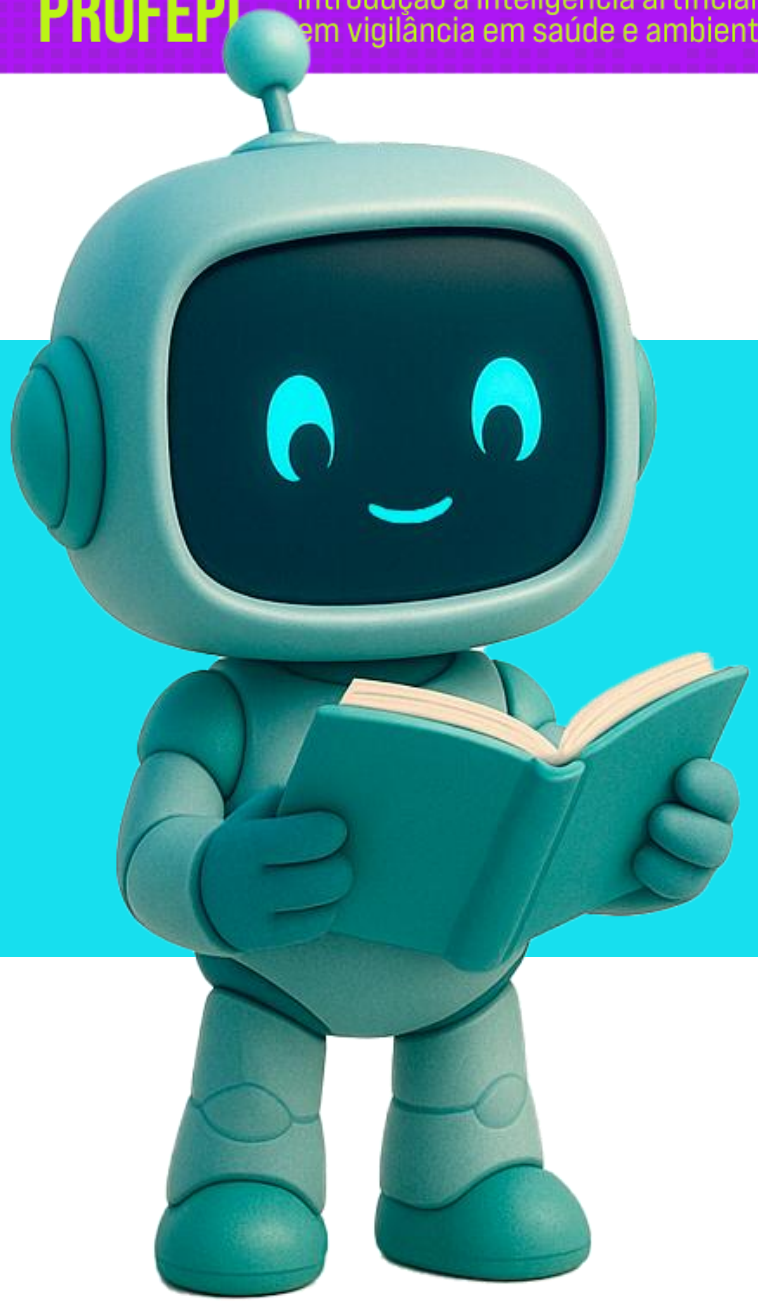
- Melhor para grandes conjuntos de dados
- Estratégia mais flexível
- Pode ter maior profundidade

Revisando...

🌳 Estrutura simples
🔄 Random Forest, XGBoost, LightGBM, CatBoost
⚖️ Equilíbrio entre desempenho e interpretabilidade
💡 Importância da validação e explicabilidade
🏥 Aplicações reais na saúde e vigilância epidemiológica

Revisando...

📊 Sempre avaliar a qualidade dos dados antes do modelo
🧪 Testar diferentes algoritmos e comparar métricas
🔍 Usar ferramentas de interpretabilidade (ex.: SHAP, LIME)
📚 Manter-se atualizado com novas bibliotecas e métodos
🤝 Integrar conhecimento técnico com contexto da área da saúde



O valor de um algoritmo não está apenas na sua precisão, mas no quanto ele contribui para decisões mais justas, rápidas e eficazes.

Obrigada!

Keisyanne de Araujo Moura

@labdaps

keisyannemoura@usp.br

